

信号与系统实验 实验报告

组别：6

组员：信息 2301 杜斯博 2234112313

信息 2301 敖凯航 2234214025

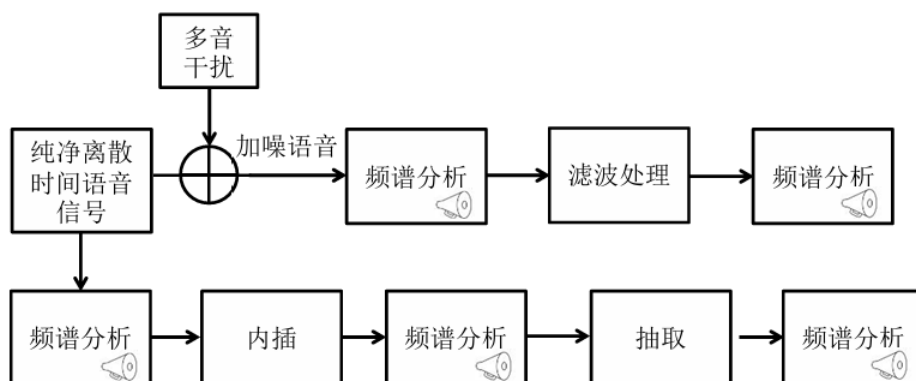
语音信号的频谱分析与频域干扰抑制

一、实验目的

语音信号是最常见的信号之一，对其进行研究具有重要的理论意义和实用价值。对语音信号进行频谱分析是语音信号处理中常见的方法。在本实验中，我们将对离散时间语音信号样本进行频谱分析，观察语音信号的频谱特性，并根据语音信号的频谱特点设计离散时间滤波器进行干扰抑制处理，并同时考察经过相应处理后的语音信号的特性。与此同时，本实验还将对语音信号进行抽取和内插等处理。通过本实验可以了解语音信号的特性，加深对傅里叶分析的理解。

二、实验简介

本实验对已录制的语音信号样本进行频谱分析及处理。给定两个离散语音信号 `trip.wav` 和 `6noise.wav`（采样率为 22.05 KHz），分别对应同一个语音信号的纯净无干扰版本和叠加了两个单音正弦干扰的版本。将加扰语音基于频域分析进行干扰抑制处理后通过扬声器播放。人耳可以感知滤波处理后语音信号发生的相应变化。本实验还将对语音信号进行抽取和内插等处理，实现基本的变速率处理。整个实验的原理框图如图所示。



图表 1 实验原理简介

三、实验内容

(1) 通过扬声器感知纯净语音与加扰语音样本之间的差异。利用 MATLAB 编程，编写绘制信号频谱特性的子函数，绘制语音信号样本的幅频特性，比较纯净语音与加扰语音样本的频谱差异。

经过音频试听，可以发现纯净的音频为男声 “Enjoy your trip.” 而加噪音频只能听出刺耳的噪声，无法分辨音频内容。

实验代码：draw_afc_gpu(x) 用于绘制信号图谱。在编写代码时，考虑到运行速度而使用 gpu 编码。

```

function X_x = draw_afc_gpu(x)
% DRAW_AFC_GPU : Draw amplitude-frequency characteristic using GPU
acceleration

x_length = length(x);
w = gpuArray(-pi:0.0002*pi:pi);
n = gpuArray((1:x_length)');
x_gpu = gpuArray(x);

% 预分配
X_x_gpu = gpuArray.zeros(size(w));

% 使用 GPU 并行计算
for k = 1:length(w)

```

```

    exponent = exp(-1j * w(k) * n);
    X_x_gpu(k) = dot(x_gpu,exponent); %点积
end

% 将结果移回 CPU 内存
X_x = gather(X_x_gpu);

plot(gather(w), 20*log10(abs(X_x)));
xlabel("\omega_{q}");
ylabel("20lg|X(e^{\omega_q})|");
title('Amplitude-Frequency Characteristic (GPU Accelerated)');
grid on;

```

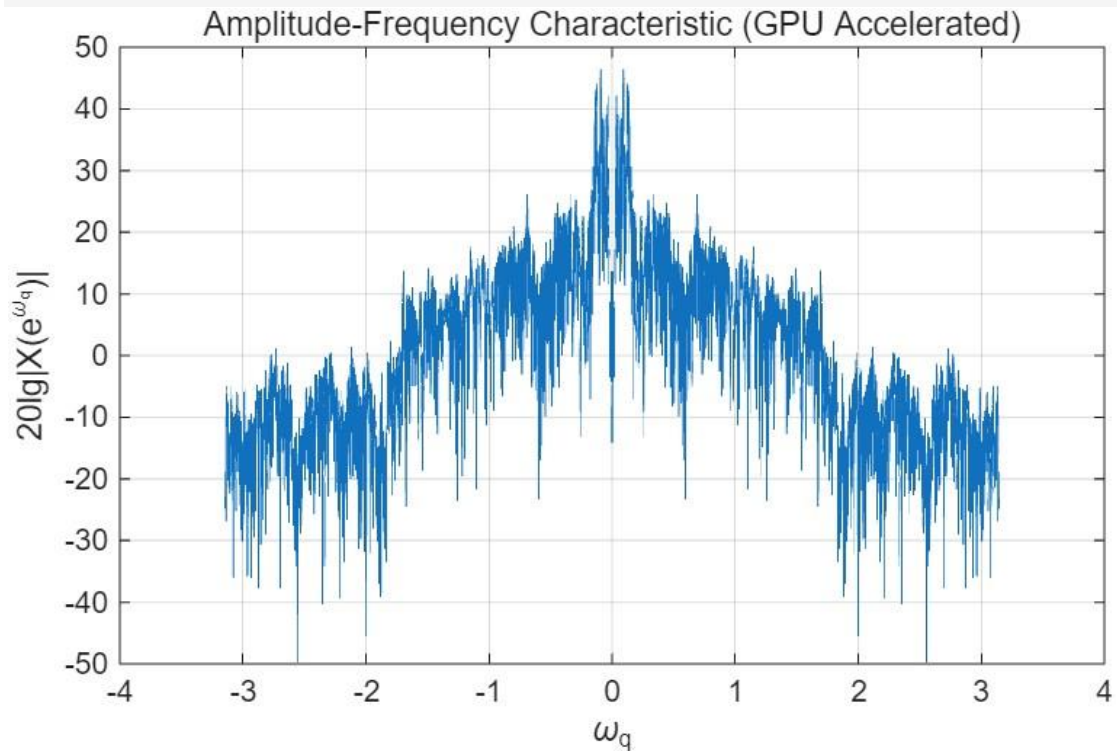
绘制纯净信号和加噪信号的频谱图实验结果如下：

纯净语音的幅频特性

```

tic;
[x1,fs1] = audioread('trip.wav');
x1=mean(x1,2);
player_x1=audioplayer(x1,fs1);
draw_afc_gpu(x1);
toc;

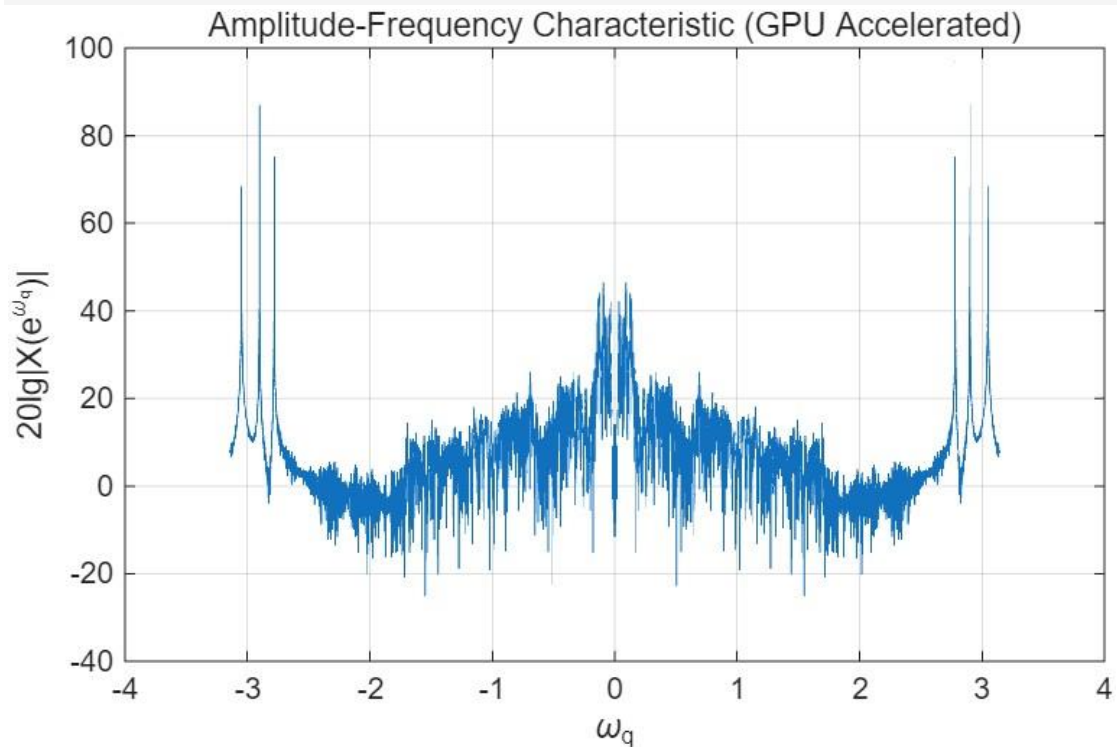
```



图表 2 纯净语音信号的幅频特性

加扰语音的幅频特性

```
tic;
[x2,fs2] = audioread('6noise.wav');
x2=mean(x2,2);
player_x2=audioplayer(x2,fs2);
draw_afc_gpu(x2);
toc;
```



图表 3 加扰语音信号的幅频特性

(2) 设计离散时间滤波器，对加扰语音信号进行滤波处理，实现干扰抑制。观察干扰抑制后的语音信号频谱，并用人耳感知干扰抑制效果。(FIR 陷波滤波器设计可以通过在单位圆设置零点方式来实现)

通过对比纯净语音信号频谱和加噪语音信号频谱，可以发现三个噪音频率位于 $\omega_p=3$ 的附近。设计三个滤波器后对其进行级联和增益保持后，与加噪信号进行卷积即可滤除噪声。

实验代码及频谱图如下：

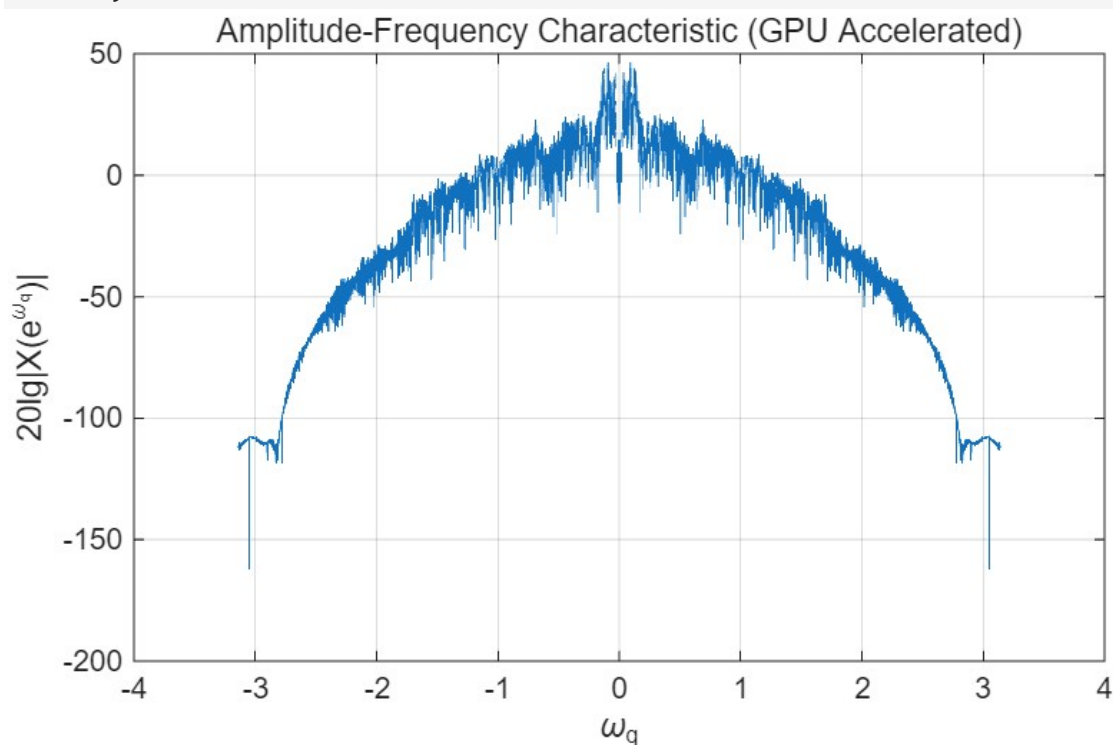
离散时间滤波器实现干扰抑制

```
w1=2.78094;h1=[1,-2*cos(w1),1];
```

```

w2=2.90095;h2=[1,-2*cos(w2),1];
w3=3.04923;h3=[1,-2*cos(w3),1];
h=conv(h1,h2);h=conv(h,h3);
h=h/sum(h);
x2_fliter=conv(x2,h);
player_x2_fliter=audioplayer(x2_fliter,fs2);
tic;
draw_afc_gpu(x2_fliter);
toc;

```



图表 4 滤波抑制干扰后信号频谱

可以观察到噪声基本被滤除，通过扬声器播放滤波后语音可以清楚的听出男声“Enjoy your trip.”

(3) 对纯净的语音信号首先进行 2 倍内插实现增采样，然后进行 5 倍抽取实现减采样，其中内插滤波器分别采用零阶保持滤波器以及时间截断的理想低通滤波器。观察内插和抽取过程中的信号频谱的变化。采用合适的采样速率播放语音信号，感知处理过程中的语音效果。

1. 零阶保持内插和抽取过程：采用 `repelem` 函数实现零阶内插增采样，`downsample` 函数实现抽取减采样。

代码如下：

```
%0 阶保持内插
```

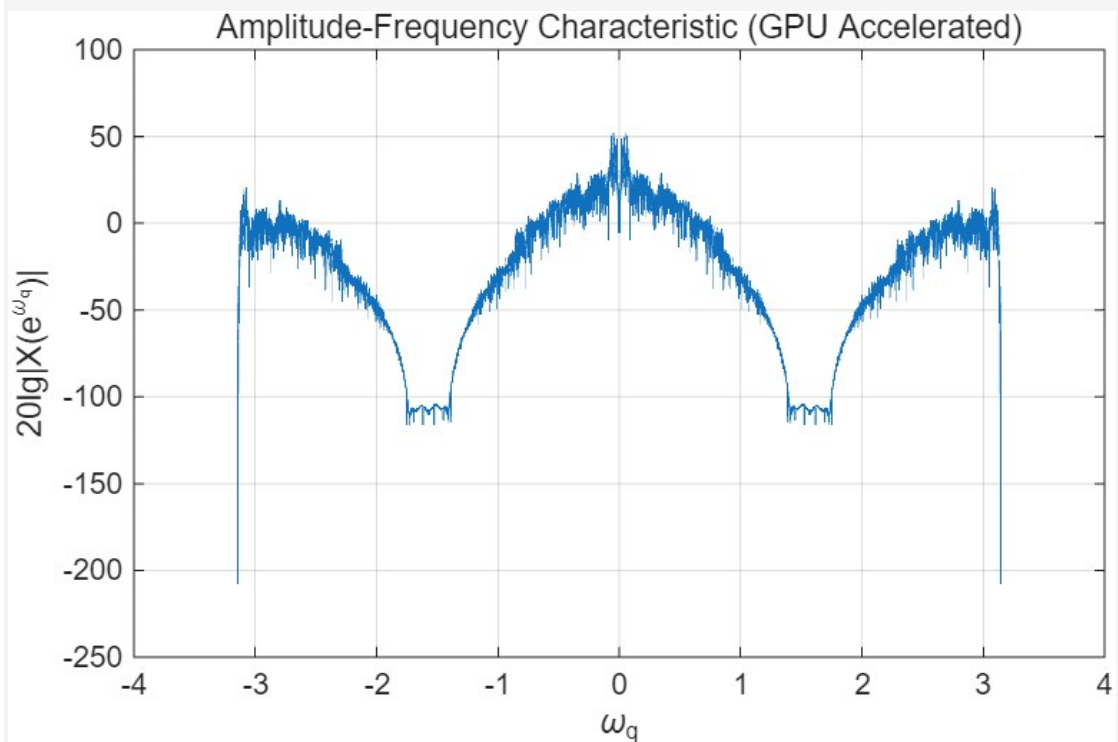
```
x3=repelem(x2_fliter,2);
```

```
player_x3=audioplayer(x3,fs2*2);
```

```
tic;
```

```
draw_afc_gpu(x3);
```

```
toc;
```



图表 5 零阶保持 2 倍内插后的信号图谱

```
%抽取
```

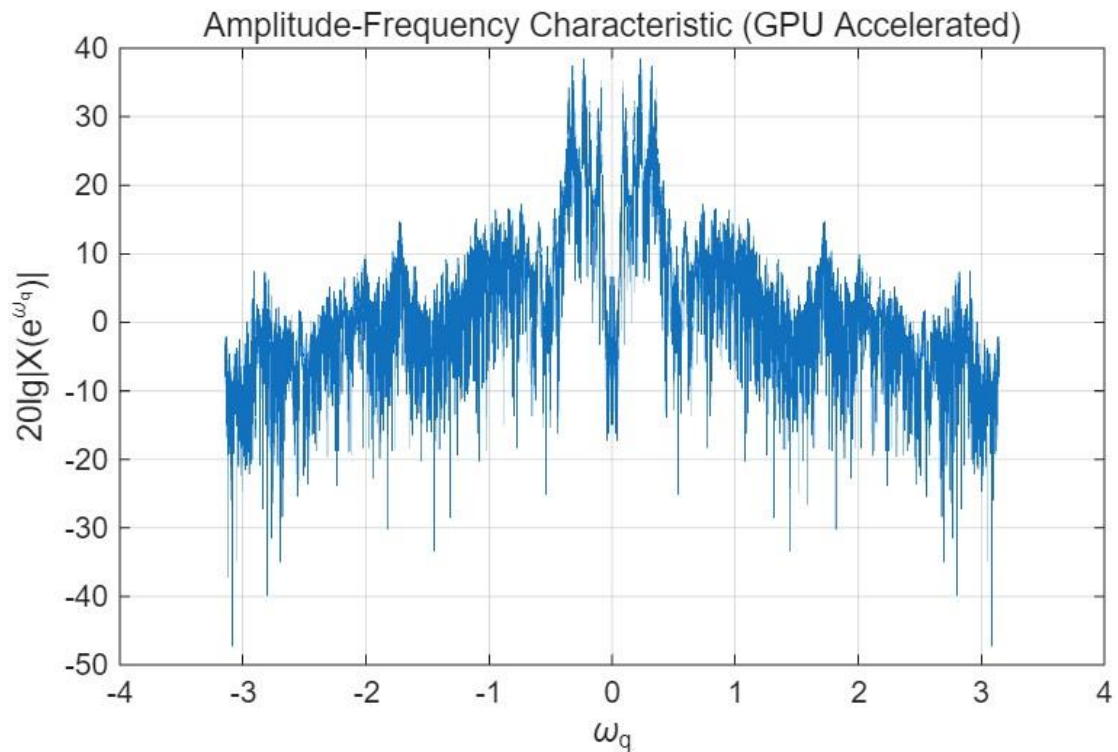
```
x3_5=downsample(x3,5);
```

```
player_x3_5=audioplayer(x3_5,fs2*2/5);
```

```
tic;
```

```
draw_afc_gpu(x3_5);
```

```
toc;
```



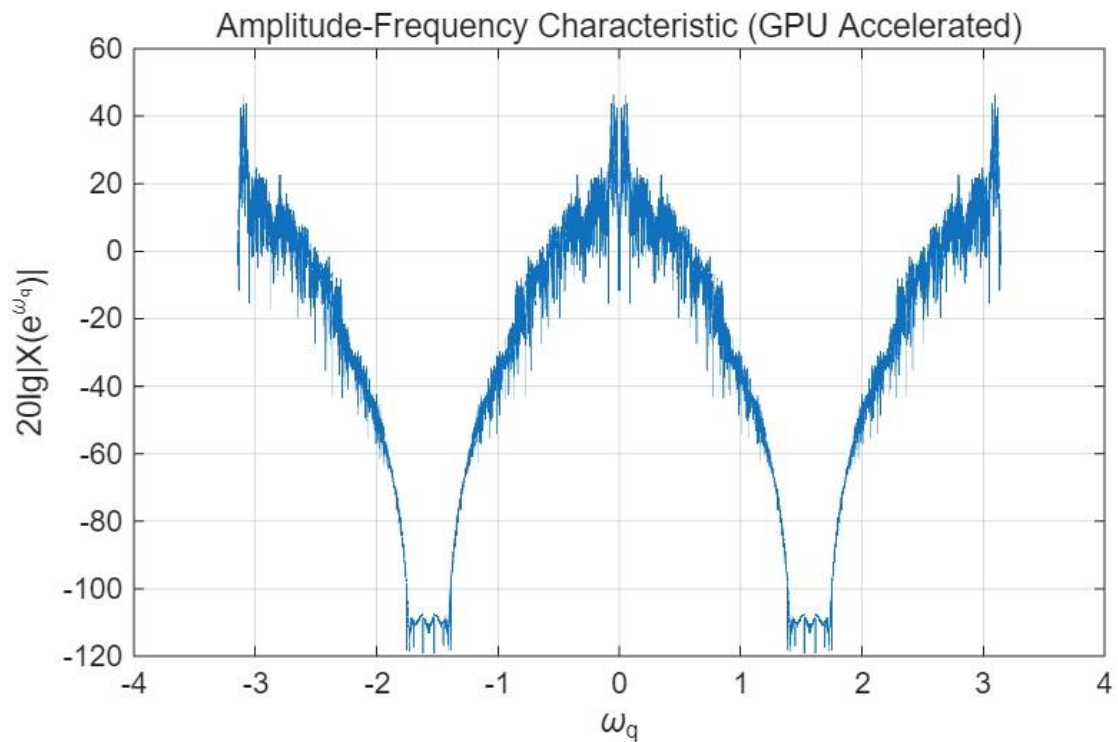
图表 6 零阶保持内插并抽取后的信号图谱

采用合适的采样率进行播放时，内插跟抽取后都与原纯净信号播放效果无明显差异。

2. 时间截断的理想低通滤波器内插和抽取过程：使用 upsample 函数补 0 增加采样率之后，设计低通滤波器和抽取过程。

代码如下：

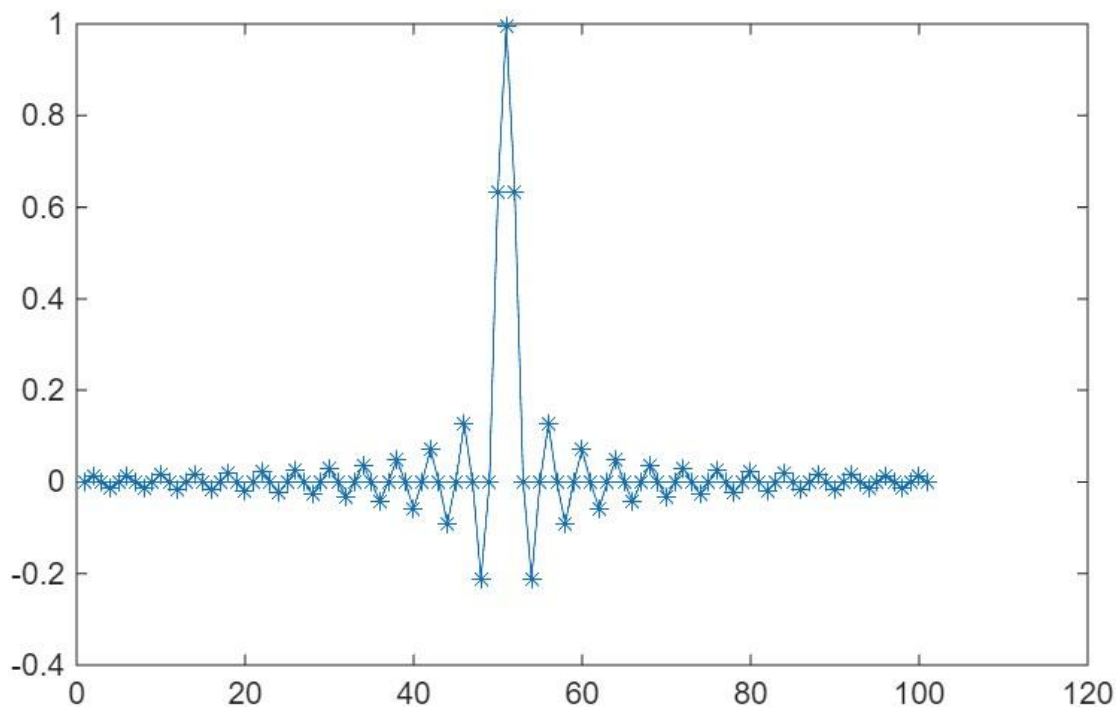
```
%时间截断的理想低通滤波器  
%补 0，增加采样率  
x4=upsample(x2_fliter,2);  
tic;  
draw_afc_gpu(x4);  
toc;
```



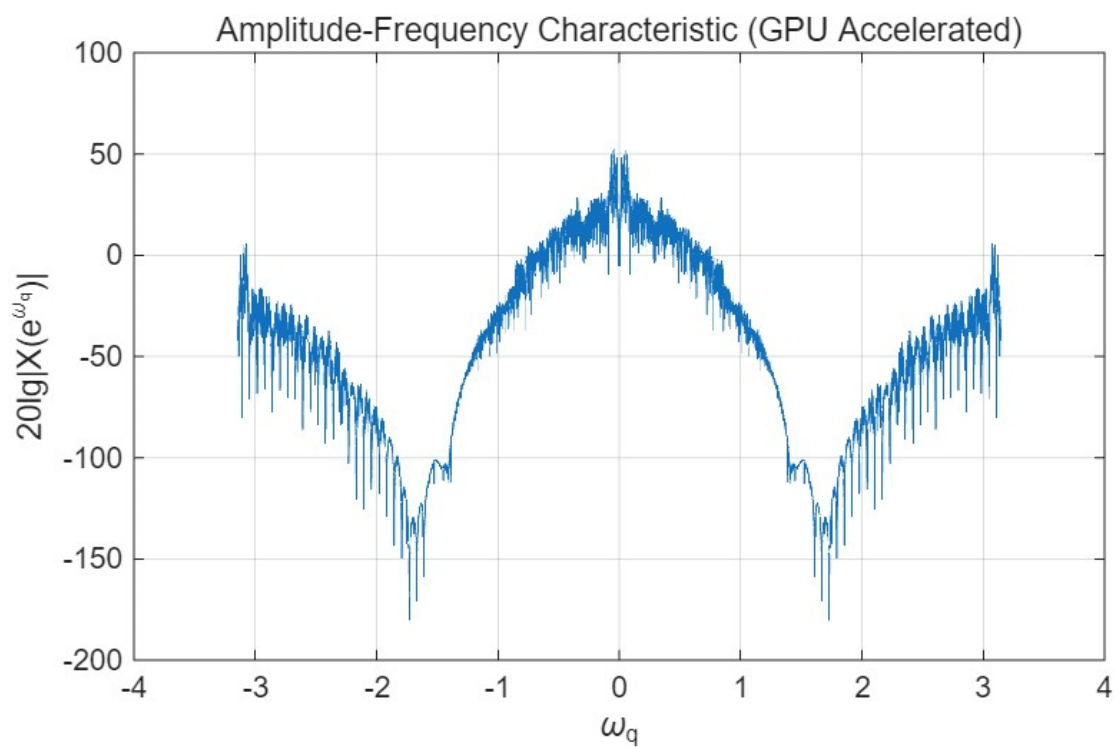
图表 7 信号补 0 后频谱

```
%设计低通滤波
h4_length=101;
h4=zeros(h4_length,1);
center = (h4_length + 1) / 2;

for i = 1:h4_length
    t = i - center;
    if abs(t) < eps
        h4(i) = 0.5;
    else
        h4(i) = sin((pi/2) * t) / (pi * t);
    end
end
h4=h4/sum(h4);
h4=h4*2;
plot(h4,'*-');
x4=conv(x4,h4);
player_x4=audioplayer(x4,fs2*2);
tic;
draw_afc_gpu(x4);
toc;
```

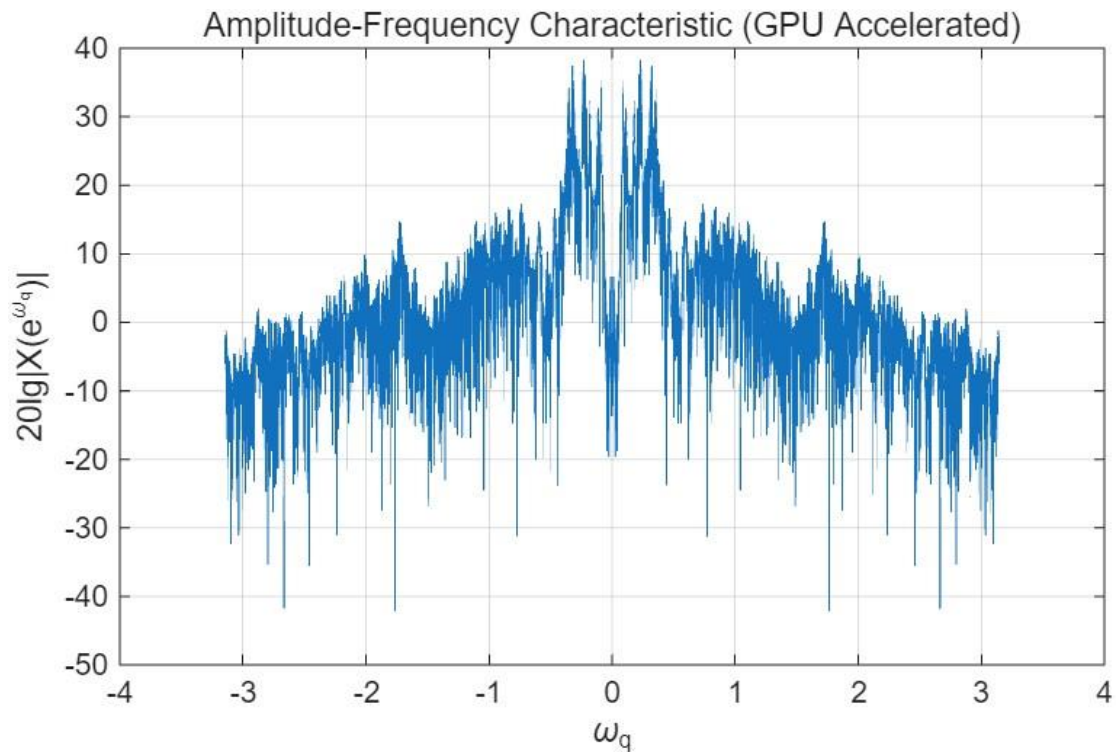



图表 8 时间截断的理想低通滤波器时域特性



图表 9 时间截断理想低通滤波器内插后信号频谱

```
%抽取
x4_5=downsample(x4,5);
player_x4_5=audioplayer(x4_5,fs2*2/5);
tic;
draw_afc_gpu(x4_5);
toc;
```



图表 10 时间截断理想低通滤波器内插抽取后信号频谱

采用合适的采样率进行播放时，内插跟抽取后都与原纯净信号播放效果无明显差异。

附注：用于步进运行调试的播放代码：

```
%{
    play(player);
    pause(player);
    resume(player);
    stop(player);
    isplaying(player);
}%
% 需要步进 按 F10
%经过滤波后的纯净音频
play(player_x2_fliter);
stop(player_x2_fliter);
%0 阶保持内插后的音频
play(player_x3);
stop(player_x3);
%抽取
play(player_x3_5);
stop(player_x3_5);
%低通滤波后的音频
play(player_x4);
```

```
stop(player_x4);  
%抽取  
play(player_x4_5);  
stop(player_x4_5);
```

四、思考题

(1) 语音信号在离散时间域的频率与连续时间域的频率是如何对应的？试判断叠加的单音干扰对应的连续时间频率。

根据采样规律可知连续时间域的频率与离散时间域的频率关系为 $f = \frac{\omega \cdot f_s}{2\pi}$

$$\omega_1 = 2.78094, f_1 = 9760\text{Hz}$$

$$\omega_2 = 2.90095, f_2 = 10180\text{Hz}$$

对所加三个单音干扰，有 $\omega_3 = 3.04923, f_3 = 10700\text{Hz}$

(2) 如果给语音样本再叠加随机的白噪声信号，其频谱分布在整个频段，此时的干扰抑制滤波器应如何设计？

白噪声指的是包含所有频率分量的噪声，且这所有的频率分量是等值的，其功率谱密度在所有频率上都是一个常数。尽管白噪声是具有均匀功率谱的平稳随机过程，当它通过线性系统后，其输出端的噪声功率就不再均匀。

在滤除白噪声时，可以考虑经 LTI 系统使输出端的噪声功率集中在固定频带处，再用陷波滤波器滤除，再经 LTI 系统还原信号。

(3) 通过内插抽取处理可以调整语音信号采样速率。试针对给定的纯净语音信号样本，在不发生频谱混叠条件下，通过变速率处理尽可能得到采样率最低的样本序列。

由图 2 纯净语音频谱可知，能量主要集中于 5kHz 以下 ($\omega \approx 1.43\text{rad}$)。根据

奈奎斯特定理，最低采样率需大致需满足 采样率大于 10kHz 。选择采样率 11.025kHz ，为原采样率的 $1/2$ 。则先对信号内插实现 2 倍增采样，再进行 4 倍抽取减采样，实现采样率减半。

(4) 你在实验中发现了什么问题，试用掌握的理论知识对其做出分析和讨论。

1. CPU 运行速度较慢，在课下模拟仿真时，对时长 16s 的音频进行 10000 的采样率频谱分析需要 $1\text{m}30\text{s}\sim 2\text{mins}$ ，不利于根据结果反复调试代码。改用 GPU 运行后速度大幅提升，仅需要不到 10s 。

2. 在设计时间截断的理想低通滤波器进行 2 倍内插时，忘记对时域特性的 sinc 函数乘以 2 和归一化，导致信号内插语音播放声音强度减弱；

在最初设计时间截止的低通滤波器时，时间限制给到了 1000 ，但实际上造成了浪费。大于 100 时信号已经基本趋于 0，过高的时间限制造成了无效的资源浪费。

(5) 通过实验你有哪些收获，对进一步改进实验有什么建议。

通过本次实验，加深了我们对于语音信号频谱分析和采样的理解。实验中我们实现了纯净语音与加扰语音的频谱对比，并设计了级联 FIR 陷波滤波器（零点配置法）基本有效抑制了噪声。在采样率变换环节，进一步熟悉了零阶保持和理想低通内插结合抽取的操作。这次实验不仅巩固了我们对傅里叶分析和滤波器设计的理论知识，更提升了我们面对工程问题的解决能力。

对于实验建议，可以再多给出可能会用到的 matlab 函数参考，比如在内插和抽取时用到的 `repelem` 函数和 `resample` 函数。